Pininfarina
Istituto Tecnico Industriale Statale

Informatica e Telecomunicazioni

TrainTrack Manager

ALUNNO
Cammarata Alessio

CLASSE
5^A info

Anno Scolastico 2023/2024

Dichiarazione di Originalità

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Abstract

In questo documento verrà mostrato e spiegato in modo completo come funziona e come è stato costituito il progetto finale, TrainTrack Manager. TrainTrack Manager è un'applicazione che permette di controllare un plastico ferroviario tramite Arduino e un'interfaccia grafica in Python. La documentazione online fornisce istruzioni dettagliate per l'installazione e l'uso, rendendo l'applicazione accessibile a tutti gli appassionati.

Indice

1 Considerazioni **1**

2 Introduzione **1**

3 Obiettivi del progetto **2**

4 Metodologia **2**

5 Chiarimenti **3**

5.1 Funzionamento della parte di progetto riguardante Arduino 3

5.2 Funzionamento dell'Interfaccia grafica 4

5.3 Funzionamento del sistema di automazione 4

6 Arduino **5**

6.1 Parte Fisica legata ad Arduino 5

6.1.1 Arduino Mega 2560 5

6.1.2 Arduino Uno Rev 3 7

6.2 Parte Logica legata ad Arduino 8

6.2.1 Arduino Mega 2560 8

6.2.2 Arduino Uno Rev 3 8

7 Interfaccia Grafica **10**

7.1 Motivazione della Scelta di Python 10

7.2 Struttura del Programma 10

7.3 Funzionalità della GUI 11

7.4 Progettazione della GUI 12

7.4.1 Processo di Progettazione 12

7.5 Tecnologie Utilizzate 13

7.6 Dipendenze Logiche tra i File 15

7.7 Struttura e funzionamento dell'app 16

7.7.1 Pagina Principale 16

7.7.2 Pagina Impostazioni 19

7.7.3 Pagina Gestione Locomotive 20

7.7.4 Pagina Monitoraggio Circuiti 21

7.8 Documentazione e Contributi 23

8 Sistema di Automazione **24**
8.1 Componenti hardware & software 24
8.2 Algoritmo di Controllo 24
8.2.1 Acquisizione dei Dati dai Sensori RFID 24
8.2.2 Logica di Controllo degli Scambi 25
8.2.3 Prevenzione delle Collisioni 25
8.3 Modalità Automatica 26
8.3.1 Attivazione della Modalità Automatica 26
8.3.2 Operazioni Automate 27
8.4 Test e Ottimizzazione 28
8.4.1 Procedure di Test 28
8.4.2 Ottimizzazione 28
8.5 Considerazioni Finali 28
9 Sviluppi futuri **29**
9.1 Altri Miglioramenti 29
10 Diagramma di Gantt **30**
11 WBS **32**
12 Tabelle dei Costi **33**
Riferimenti **35**

1 Considerazioni

Il progetto nasce dalla necessità di **controllare un plastico ferroviario, con il computer**, di mio padre, essendo un progetto molto scalabile, esso **è diviso in 3 parti:**

- Arduino
- Interfaccia grafica
- Sistema di automazione

Per quanto riguarda le **tempistiche**, ho deciso di svolgere la parte fisica riguardante il circuito del treno fino a circa metà gennaio, il resto del tempo fino a **termine scadenze** (Maggio-Giugno) è stato prettamente riservato all'avanzamento della parte Software, ma sono stati eseguiti anche controlli sul plastico per assicurarsi il mantenimento.

2 Introduzione

Il progetto presentato ha come obiettivo la **creazione di un modello di sistema ferroviario** funzionante che **comprende più locomotive e la gestione informativa di esse;** È stata inoltre prevista la costruzione di un **plastico ferroviario** che comprenda una stazione ed un deposito treni utilizzabili a pieno regime. Durante la costruzione del plastico sono stati aggiunti anche dei ponti che permettono di nascondere i sensori, utili al **sistema di automazione.**

Lo scopo è quello di **utilizzare le conoscenze ottenute** durante il mio percorso di studi per sviluppare un **prototipo di applicativo** che possa rispondere alle esigenze del cliente (mio padre).

3 Obiettivi del progetto

Il progetto mira a:

- **Realizzare un sistema di controllo** completo per un plastico ferroviario tramite computer.
- **Integrare Arduino** per il **controllo** fisico delle **locomotive e dei sensori.**
- Sviluppare un'interfaccia grafica intuitiva per la gestione del sistema.
- **Implementare un sistema di automazione** per il controllo intelligente delle operazioni ferroviarie.
- **Garantire la scalabilità e la modularità** del progetto per future espansioni e miglioramenti.

4 Metodologia

Per raggiungere gli obiettivi prefissati, la metodologia adottata comprende:

- **Analisi dei requisiti:** Raccolta e definizione dei requisiti hardware e software necessari.
- **Progettazione**: Pianificazione dell'architettura del sistema, suddivisa in tre componenti principali (Arduino, Interfaccia grafica, Sistema di automazione).
- **Sviluppo**: Implementazione delle componenti fisiche (Arduino) e software (Interfaccia grafica e automazione).
- **Test e Verifica:** Esecuzione di test sul sistema per verificare il corretto funzionamento di tutte le componenti.
- **Integrazione e collaudo:** Unione delle diverse componenti e collaudo del sistema integrato per assicurare l'interoperabilità.
- **Documentazione:** Creazione di documentazione dettagliata sul progetto, includendo manuali d'uso e report tecnici.

5 Chiarimenti

In questa sezione sono definiti un po più nel dettaglio i **vari settori del progetto**, essendo esso composto da **vari componenti**, i tre **sottoprogetti** e i **componenti** e le **conoscenze necessarie** sono descritti singolarmente.

5.1 Funzionamento della parte di progetto riguardante Arduino

La parte del progetto riguardante **Arduino** è atta alla realizzazione di un **plastico di ferromodellismo domotico controllato da Arduino**;
I materiali necessari alla produzione del modello sono stati:

- Arduino Mega 2560
- Arduino Uno Rev 3
- Motor Shield Rev 3
- Modello su tavola di Circuito Ferroviario
- 8 Relè con rotella da 18V
- Scheda prestampata di 8 relè 5V optoisolata
- Cavetteria varia
- 8 Lettori Rfid MFRC522 e relativi tag a tessera
- Diodi e led polarizzati

Le conoscenze necessarie in questa parte di progetto sono dunque le seguenti:

- Buona conoscenza dell'**elettronica** e della creazione di **circuiti non complessi**
- Ottima conoscenza di **Arduino** e della **creazione di sketch** appositi ai sensori
- Conoscenza adeguata al mondo del modellismo per quanto concerne la realizzazione di edifici e percorsi del plastico

5.2 Funzionamento dell'Interfaccia grafica

La parte del progetto riguardante **l'Interfaccia grafica** si concentra sullo sviluppo del sistema di controllo delle locomotive.
Per questo obiettivo, ho deciso di utilizzare l'interfaccia grafica di python.
Le conoscenze necessarie sono dunque le seguenti:

- Conoscenza adeguata del **linguaggio Python**
- Conoscenza adeguata dei **vari costrutti e delle librerie**:
  - Pyserial
  - Opencv
  - Os
  - Tkinter
  - Threading
  - Queue
  - Pillow
  - Subprocess
- Buona conoscenza dei **metodi di comunicazione di Arduino**

5.3 Funzionamento del sistema di automazione

Questa parte del progetto ha come obiettivo la **creazione di un algoritmo per la gestione dei treni senza doverli controllare manualmente**, evitando gli scontri.
L'automatizzazione del circuito viene garantita tramite l'utilizzo di **relè e di lettori e tag rfid.**
Questi ultimi andranno **montati sulle locomotive**, ognuna dovrà avere il proprio tag, tramite l'interfaccia è possibile registrare il tag della locomotiva in memoria e dunque utilizzarla liberamente nel circuito durante la modalità automatizzata.
I lettori avranno il compito di **segnalare il passaggio di una locomotiva** e saranno dunque posizionati in siti sensibili in modo da dare al processo in esecuzione le migliori informazioni possibili per gestire gli scambi.
I relè hanno il **compito di alternare la posizione dei cambi**, verranno **controllati in automatico dall'algoritmo** che a seconda delle informazioni deciderà se scambiare o no il deviatoio.

6 Arduino

Questa parte del progetto è stata divisa per ragioni logistiche in **fisica e logica.**

6.1 Parte Fisica legata ad Arduino

La parte **fisica** riguarda **Arduino**, il suo utilizzo nel circuito e i vari **circuiti elettrici** che abbiamo creato per permettere la **gestione dei deviatoi e dei sensori**. Ogni Arduino ha il suo circuito e viene utilizzato per diversi scopi.

6.1.1 Arduino Mega 2560

La parte Fisica legata ad Arduino Mega 2560 si compone di:

- **Locomotive**
  Arduino Mega 2560 è l'arduino che **gestisce la corrente modulata** che deve arrivare ai **decoder**; la corrente in questione passa attraverso il **motor shield Rev3** montato su arduino che permette di gestire la **modulazione** delle entrate della corrente, attraverso la libreria DCCpp di cui parlerò nella sezione successiva, in modo da **nascondere dei messaggi nella corrente** che può passare **attraverso i binari** e arrivare ai **decoder** posizionati all'interno di ogni locomotiva, i quali **traducono** i pacchetti ed **eseguono** le azioni.

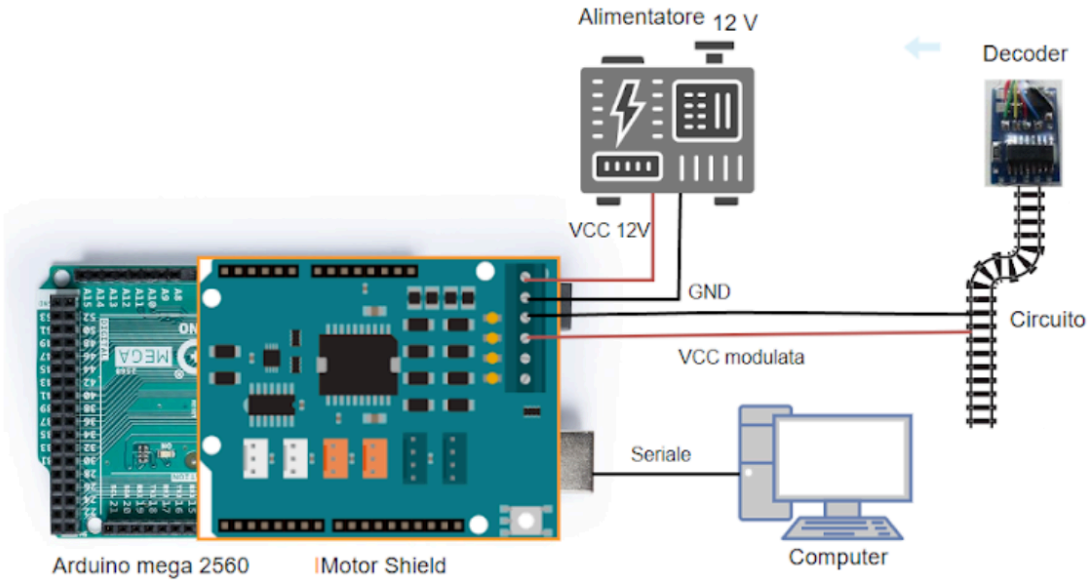


Figura 1: Logica dei collegamenti per il controllo delle locomotive con Arduino Mega 2560

- **Deviatoi**
  Oltre a gestire le locomotive Arduino Mega 2560 gestisce l'attività dei deviatoi. Esso infatti è in grado di mandare degli impulsi di 5 V dai pin digitali, in grado di azionare i relè che controllano gli scambi. Questi collegamenti tra arduino e relè necessitano un isolamento fisico, per questo abbiamo utilizzato una scheda prestampata di 8 relè 5V optoisolata che permette di attivare i relè tramite arduino e di isolare i due circuiti evitando interferenza; è stato inoltre necessario isolare il circuito relativo ai relè tramite dei diodi e dei led polarizzati che assorbono i segnali di ritorno causati dalla bobina dei relè.

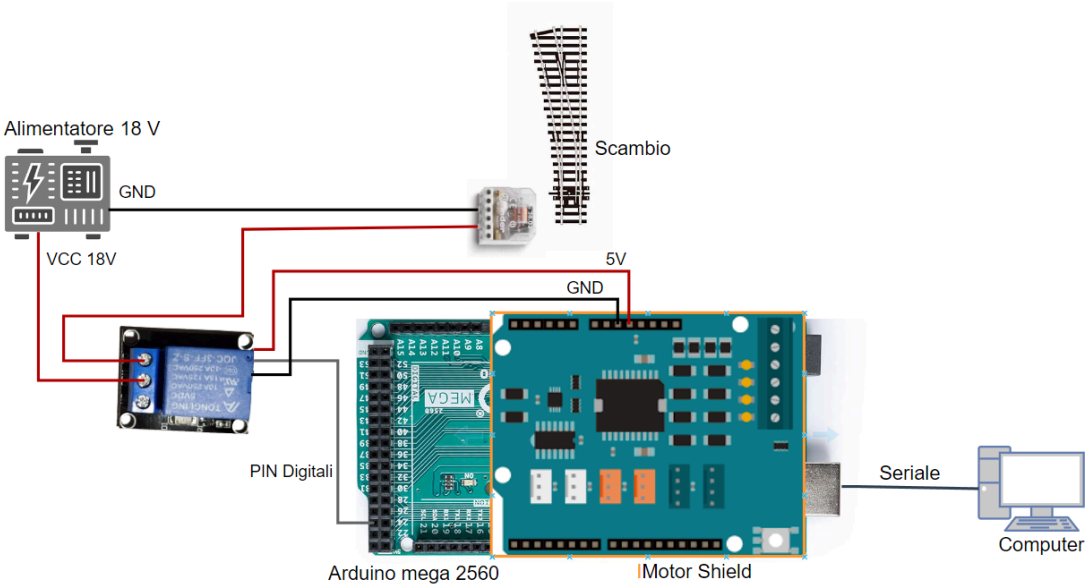


Figura 2: Logica dei collegamenti per il controllo delle locomotive con Arduino Mega 2560

6.1.2 Arduino Uno Rev 3

I collegamenti relativi ai **sensori RFID** sono effettuati con **Arduino UNO Rev 3**, esso **gestisce tutti e 8 i sensori** del plastico e quando riceve un messaggio lo manda direttamente al computer tramite la seriale, il quale lo processa nell'algoritmo. Per la realizzazione dei collegamenti con Arduino sono stati usati i Pin dedicati alla **comunicazione SPI** e cavi di diametro tra i 2 e i 3 mm per ridurre le cadute di tensione.

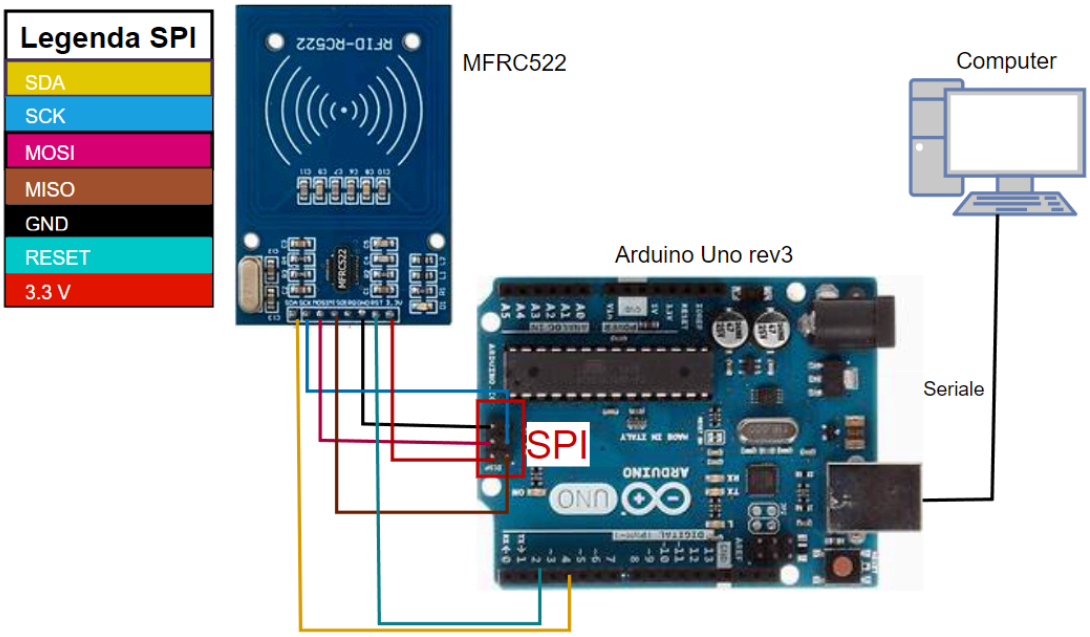


Figura 3: Logica dei collegamenti con i sensori MFRC522 per Arduino Uno Rev 3

In allegato lascio la documentazione tecnica relativa ai circuiti.

- Documentazione tecnica sui circuiti [1]

6.2 Parte Logica legata ad Arduino

La parte **logica** comprende i codici in **linguaggio c++ degli sketch** di arduino che servono sia per far muovere le locomotive **(Arduino Mega 2560)** sia registrare la loro posizione **(Arduino UNO).**

6.2.1 Arduino Mega 2560

Bisogna specificare che lo sketch utilizzato in questo Arduino è la **libreria DCCpp** che si occupa della gestione delle locomotive tramite **decoder**, quest'ultima è **Open Source** e facilmente implementabile.
Il processo che esegue questa libreria è quello di **rimodulare** la corrente alternata che arriva dal generatore in modo da nascondere ,all'interno, dei comandi che il decoder **traduce ed esegue** sulle locomotive, ciò permette di dare la corrente ai treni tramite i binari senza l'ausilio di cavi volanti che disturberebbe la vista.

In particolare questa libreria definisce anche la **gestione in memoria delle informazioni e i vari comandi** che l'Arduino deve ricevere per fare certe azioni. In allegato sono disponibili maggiori dettagli sulla libreria.

- Libreria DCCpp [2]

6.2.2 Arduino Uno Rev 3

Lo sketch utilizzato in questo Arduino utilizza invece la **libreria MFRC522 di Miguel Balboa**, e si concentra sulla gestione dei sensori, anch'essa Open Source.
Questo codice permette di **gestire i segnali dei sensori** e di **mandare messaggi personalizzati tramite seriale** al computer che li processa. In particolare, questo sketch è **scalabile e permette di cambiare il numero di sensori** a seconda di quanti se ne necessiti.
In allegato sono disponibili maggiori dettagli sulla libreria.

- Libreria MFRC522 [3]

Esempio di codice personalizzato:

```
#include <SPI.h>
#include <MFRC522.h>

#define RST_PIN         2
#define NR_OF_READERS   8

byte ssPins[] = {4, 5, 6, 7, 8, 9, 10, 18};
MFRC522 mfrc522[NR_OF_READERS];   // Create MFRC522 instance.

void setup() {
    Serial.begin(115200); // Initialize serial communications with the PC
    while (!Serial);      // Do nothing if no serial port is opened
    SPI.begin();          // Init SPI bus

    // Init each MFRC522 card
    for (uint8_t reader = 0; reader < NR_OF_READERS; reader++) {
        mfrc522[reader].PCD_Init(ssPins[reader], RST_PIN);
        delay(100);
    }
}

void loop() {
    for (uint8_t reader = 0; reader < NR_OF_READERS; reader++) {
        // Look for new cards
        mfrc522[reader].PCD_Init();

        // Controls if a card is ready to connect
        if (mfrc522[reader].PICC_IsNewCardPresent() &&
mfrc522[reader].PICC_ReadCardSerial()) {
            // Print the specific message
            Serial.print(reader + 1);
            Serial.print("/");
            dump_byte_array(mfrc522[reader].uid.uidByte, mfrc522[reader].uid.size);
            Serial.println();
            Serial.flush();
            break; // Exit from the cycle to restart it faster
        }
    }
}

// Helper routine to dump a byte array as hex values to Serial.
void dump_byte_array(byte *buffer, byte bufferSize) {
    for (byte i = 0; i < bufferSize; i++) {
        Serial.print(buffer[i], HEX);
    }
}
```

7 Interfaccia Grafica

L'interfaccia grafica ha l'obiettivo di semplificare l'utilizzo delle locomotive ed è aperta al pubblico, chiunque potrà contribuire al miglioramento di essa tramite il link della repository:

- TrainTrack Manager [4]

7.1 Motivazione della Scelta di Python

La GUI di questo progetto è stata scritta in **python**, un linguaggio familiare dato che è stato parte del programma di quarta superiore. Inoltre la partecipazione in questi anni a più di un corso di potenziamento forniti dalla scuola ha consolidato la preferenza per python ad altri linguaggi come java o c.

7.2 Struttura del Programma

Il programma di gestione della GUI è stato scritto utilizzando la **OOP**.
In particolare, esistono **sei diverse classi** che organizzano la funzionalità della GUI:

- **App:**
  Questa classe è il punto d'ingresso principale del programma. Si occupa dell'inizializzazione dell'applicazione e del coordinamento delle altre classi.
- **GUI:**
  Gestisce la costruzione dell'interfaccia utente, definendo il layout e gli elementi grafici come finestre, pannelli, etichette, e campi di input.
- **Buttons:**
  Contiene la logica per la creazione e la gestione dei pulsanti dell'interfaccia. Ogni pulsante ha un'azione specifica associata.
- **Circuit_window:**
  Gestisce la finestra che visualizza lo stato del circuito, mostrando la posizione delle locomotive e permettendo il controllo manuale degli scambi.

È anche possibile avviare la modalità automatica per un'esperienza più fluida e interattiva.

- **Algorithm:**
  Implementa gli algoritmi necessari per il funzionamento dell'applicazione, come il controllo delle locomotive e l'elaborazione dei dati dei sensori.
- **Cam:**
  Gestisce le funzionalità legate alla videocamera, come la visualizzazione in tempo reale del circuito.

7.3 Funzionalità della GUI

Le principali funzionalità dell'Interfaccia Grafica sono:

- **Gestione dei Treni:**
  Dalla GUI è possibile tenere in memoria i treni da utilizzare, gestire la loro presenza in una tabella, e modificare i dati se necessario.
- **Controllo delle Locomotive:**
  È possibile controllare i treni, gestire la velocità e la direzione con un semplice click.
- **Visualizzazione della Posizione:**
  Aprendo la finestra di controllo, è possibile visualizzare la posizione dei treni sulle rotaie e comandare manualmente gli scambi.
- **Modalità automatica:**
  È disponibile anche la modalità automatica che permette di gestire in automatico il circuito, controllando gli scambi attraverso un algoritmo ed evitando lo scontro tra le locomotive. Questa modalità supporta al minimo 2 locomotive e al massimo 3.
- **Supporto multilingua:**
  Sono disponibili fino a quattro linguaggi:
  - Spagnolo
  - Francese
  - Italiano
  - Inglese

7.4 Progettazione della GUI

Questa sezione del progetto è quella **principale** e ha richiesto la **maggior parte del tempo** dedicato al progetto. La **GUI è il centro operativo** che integra tutte le componenti del sistema, permettendo **un'interazione intuitiva e efficiente con l'hardware.**

7.4.1 Processo di Progettazione

Il processo di progettazione è stato guidato da una serie di passaggi ben definiti:

1. **Definizione dei Requisiti:**
   Inizialmente, ho **identificato le esigenze principali** dell'interfaccia, considerando le **funzionalità chiave** come la gestione dei treni, il controllo delle locomotive e la visualizzazione della loro posizione.

   Ho creato una **lista dettagliata delle funzionalità necessarie** e delle **caratteristiche desiderate**, che ha servito come **guida** durante tutto il processo di progettazione e sviluppo.

2. **Struttura del Programma:**
   Il programma di gestione della GUI è stato suddiviso in diverse classi, ognuna delle quali ha svolto un ruolo specifico nell'organizzazione e nell'esecuzione delle funzionalità dell'interfaccia.

3. **Implementazione OOP:**
   Tutte le classi sono state implementate utilizzando il paradigma di programmazione orientata agli oggetti (OOP), che ha favorito una **struttura modulare e facile da mantenere.**
   Questo approccio ha facilitato **l'estensione e la manutenzione del codice nel tempo.**

4. 4 **Test e Ottimizzazione:**
   Dopo aver completato l'implementazione iniziale, ho eseguito una serie di **test** per assicurarmi che tutte le funzionalità **funzionassero correttamente.**

   Ho **ottimizzato il codice** per migliorare le prestazioni e garantire che l'interfaccia fosse **reattiva e senza ritardi.**
   Ho anche effettuato **test di usabilità** per assicurarmi che l'interfaccia fosse **intuitiva e facile da navigare.**

7.5 Tecnologie Utilizzate

Come già anticipato nella sezione Chiarimenti, queste sono le tecnologie utilizzate:

- **Tkinter:**
  Per la realizzazione dell'interfaccia grafica abbiamo utilizzato Tkinter, la libreria standard di Python per la creazione di GUI.

- **pySerial:**
  La comunicazione con Arduino UNO è stata gestita tramite la libreria **pySerial**, che permette di inviare e ricevere dati tramite la porta seriale. È infatti necessario leggere i messaggi che vengono inviati dai sensori durante l'esecuzione dell'algoritmo.

- **OpenCV:**
  Una libreria open-source specializzata nell'elaborazione di immagini e video, utilizzata nella classe cam per attivare e visualizzare i frame dalla camera principale del dispositivo.

- **Os:**
  Una libreria Python che fornisce funzionalità per **l'interazione con il sistema operativo**, molto utile per gestire i **percorsi dei file** tra cui quelli **delle immagini e dei file di testo per le traduzioni**.
  Inoltre l'utilizzo di questa libreria permette di utilizzare l'applicazione su sistemi **MacOS o Linux.**

- **Threading:**
  La libreria **Threading** in Python è stata utilizzata nel progetto per gestire la programmazione concorrente, consentendo l'esecuzione di più attività simultaneamente. In particolare, è stata impiegata nell'algoritmo per **leggere i messaggi dai sensori** e attuare azioni in base a tali messaggi, garantendo nel contempo una **visualizzazione fluida e reattiva delle modifiche attuali sulla GUI.**
  Grazie a questa capacità di eseguire operazioni in background, il programma rimane efficiente e sensibile alle interazioni dell'utente.

- **Queue:**
  Una libreria che fornisce **strutture dati per la gestione di code**, ovvero insiemi ordinati di elementi che consentono l'inserimento e l'estrazione secondo il principio "primo entrato, primo uscito".
  Utilizzata anch'essa nella gestione **dell'algoritmo** per poter leggere i messaggi in ordine **FIFO** e lo **scambio dei messaggi letti tra i vari thread**.

- **Pillow:**
  Una **libreria di elaborazione delle immagini in Python** che consente di **aprire, manipolare e salvare immagini.**
  Nel progetto è stata principalmente utilizzata per **effettuare il resize delle immagini e anche la rotazione** di immagini presenti nel disegno del circuito come i **sensori o le casette.**

- **Subprocess:**
  Una libreria che **consente di eseguire comandi del sistema operativo** da uno script Python e interagire con essi.
  In particolare ho preferito l'utilizzo della subprocess alla os, che svolge più o meno le stesse funzioni, poiché durante l'esecuzione **non necessita un terminale** ciò permette di utilizzare le stesse funzioni **senza dare problemi visivi all'utente che utilizza il file esecutivo.**

7.6 Dipendenze Logiche tra i File

Di seguito è riportata un'immagine che illustra le **dipendenze logiche tra i vari file e classi del programma**, facilitando la **comprensione della loro interazione e struttura complessiva:**

Figura 4: Struttura Logica dell'interfaccia grafica di TrainTrack Manager

7.7 Struttura e funzionamento dell'app

L'interfaccia grafica di Train Track Manager è composta da diverse pagine, ognuna progettata per offrire **funzionalità specifiche e facilitare l'interazione dell'utente** con il sistema ferroviario. Innanzitutto essendo un'applicazione, il link sul desktop dovrà essere aperto.

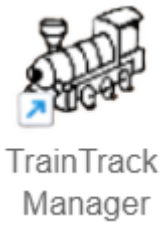


Figura 5: Immagini applicazione desktop

Di seguito vengono descritte singolarmente le principali pagine dell'interfaccia.

7.7.1 Pagina Principale

La pagina principale **è il punto di ingresso dell'applicazione**. Qui gli utenti trovano un riepilogo delle **principali funzionalità** e possono **accedere rapidamente alle altre pagine dell'interfaccia**. È inoltre possibile **cambiare la lingua del programma** tramite il menù a tendina che si apre schiacciando la bandiera in alto a destra.

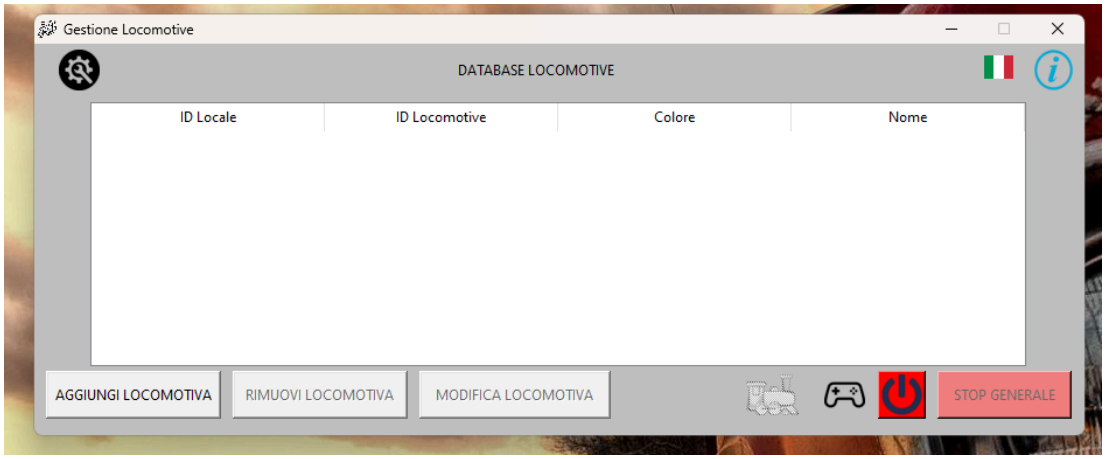


Figura 6: Pagina Principale

La pagina principale svolge anche la funzione di **gestione delle locomotive**, infatti sotto la scritta **"Database Locomotive"** è posizionata una **tabella** che si riempie tramite l'utilizzo del bottone aggiungi delle istanze delle locomotive.

Nella pagina “Aggiungi Locomotiva”, **è necessario specificare il nome della locomotiva che può essere liberamente scelto**, il **LocoID del decoder che deve essere unico** per ogni locomotiva, nel caso in cui sia è uguale **è possibile modificarlo dal tasto modifica**.
Inoltre è **necessario impostare un colore**, nel caso in cui non venga specificato, **viene assegnato in maniera casuale tra quelli disponibili**, il cui numero è strettamente correlato al numero massimo di locomotive che il sistema può sopportare.

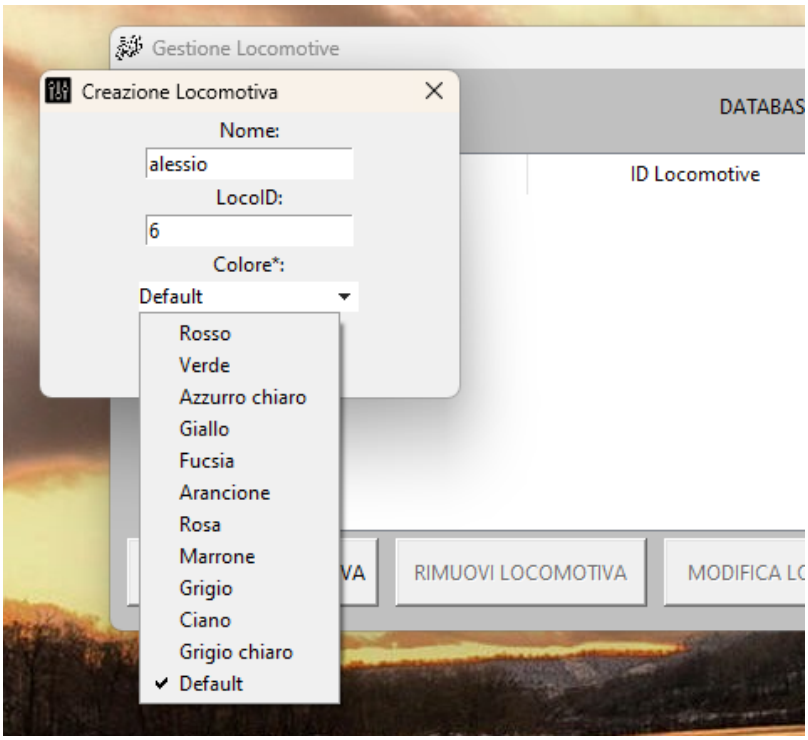


Figura 7: Finestra di aggiunta/creazione locomotive

Le istanze di treni salvate **possono poi essere visualizzate nella tabella**.

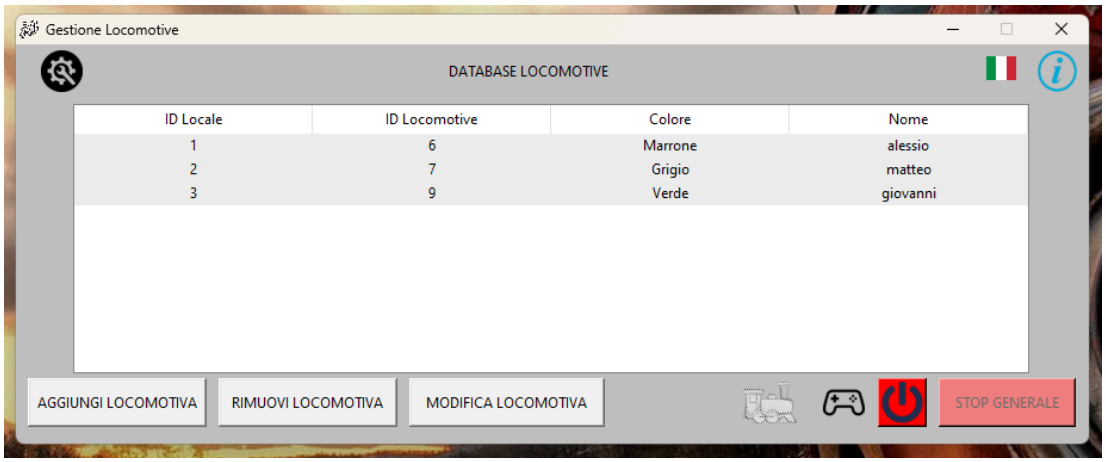


Figura 8: Pagina Principale con istanze nella tabella

È possibile poi **modificarle o rimuoverle tramite i relativi bottoni.**

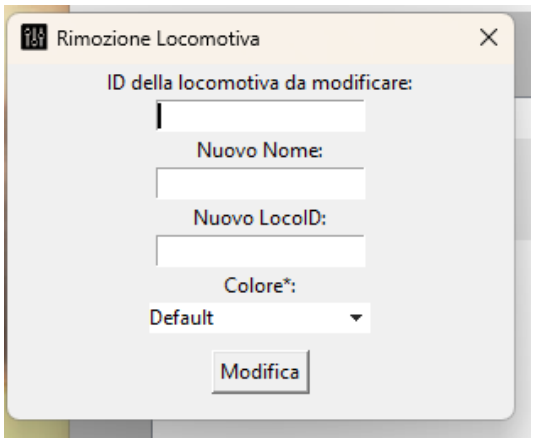


Figura 9: Finestra di modifica locomotive

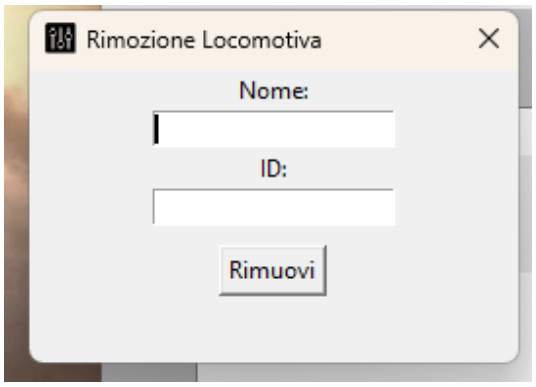


Figura 10: Finestra di Rimozione locomotive

Si può notare che alcuni bottoni **sono disattivati**, questo poiché la **corrente è disattivata**, non è infatti possibile cliccare i bottoni per controllare le locomotive o per fermare il sistema poiché la corrente non è attiva.
Per il regolare funzionamento è perciò necessario attivare la corrente premendo il tasto 'o' da tastiera o con il bottone power, che diventerà verde dopo che sarà attivato.

Figura 11: Apertura dell'interruttore della corrente

Allo stesso tempo, sarà possibile cliccare gli altri 2 bottoni, **se e solo se è presente almeno una locomotiva nella tabella.**

7.7.2 Pagina Impostazioni

La pagina delle impostazioni consente agli utenti di **personalizzare le preferenze dell'applicazione.**
È possibile **modificare il numero massimo di locomotive che può variare da 1 a 11 ed impostare le impostazioni relative alle porte seriali**, come quale porta è da **associare ai sensori e quale alla centralina DCC**, inoltre è possibile anche disabilitare le porte in caso non si vogliano utilizzare.
Nel caso in cui i dispositivi siano compatibili, sarà anche possibile visionarli nella tabella.

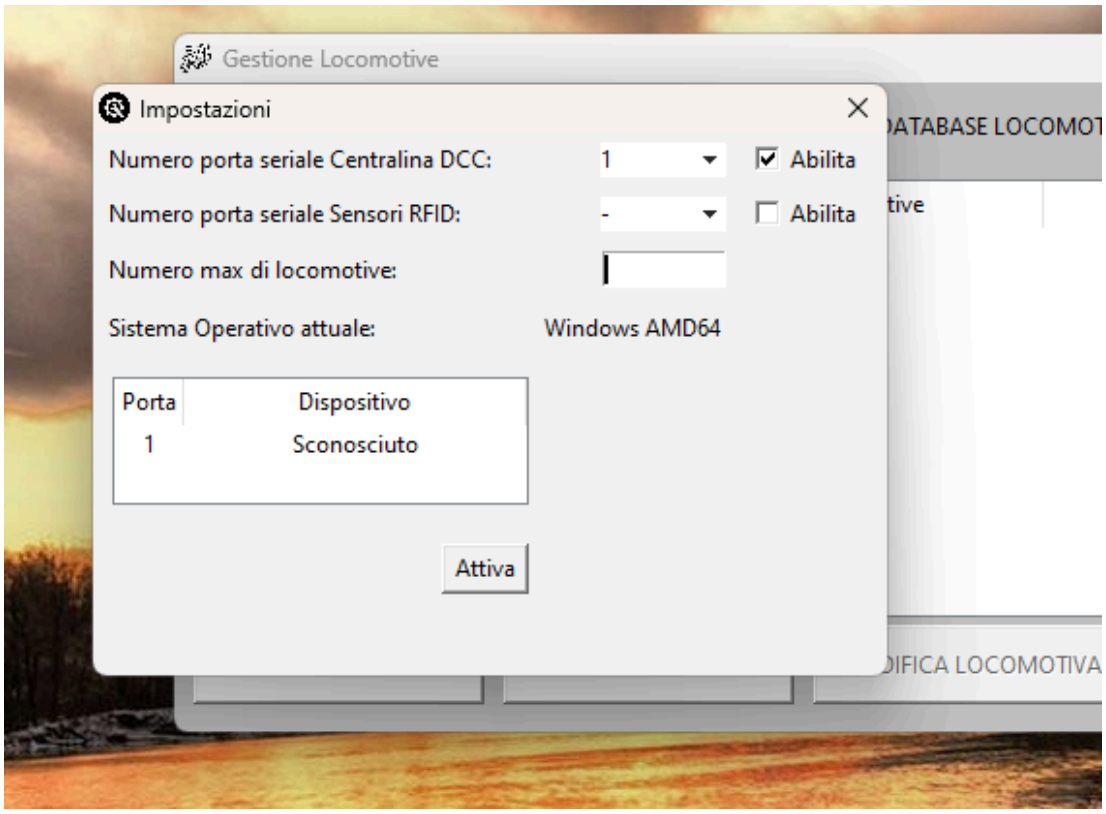


Figura 12: Finestra delle impostazioni

7.7.3 Pagina Gestione Locomotive

Questa pagina consente agli utenti di gestire le locomotive presenti nel sistema, può essere aperta tramite tastiera, premendo il numero ID della locomotiva, cliccando la locomotiva in tabella oppure tramite il menù a tendina dell'icona a forma di treno, ricordando sempre di avere attivato la corrente e di avere aggiunto in memoria almeno una locomotiva.

Figura 13: Menù a tendina per le locomotive nella pagina principale

È possibile **assegnare le loro velocità e la direzione**, è inoltre possibile **arrestare la locomotiva**, tutto ciò avviene in maniera personalizzata a seconda del LocoID della locomotiva.

Figura 14: Pagina di Controllo della locomotiva 'alessio'

7.7.4 Pagina Monitoraggio Circuiti

La finestra di **controllo e monitoraggio** del circuito fornisce una visualizzazione grafica del circuito ferroviario. Gli utenti possono **monitorare in tempo reale la posizione delle locomotive, lo stato degli scambi** ed è anche possibile **avviare la videocamera.** Attraverso questa pagina, gli utenti possono tenere traccia del flusso del traffico ferroviario e **intervenire** se necessario, **attivando gli scambi manualmente** per instradare correttamente le locomotive lungo i binari.

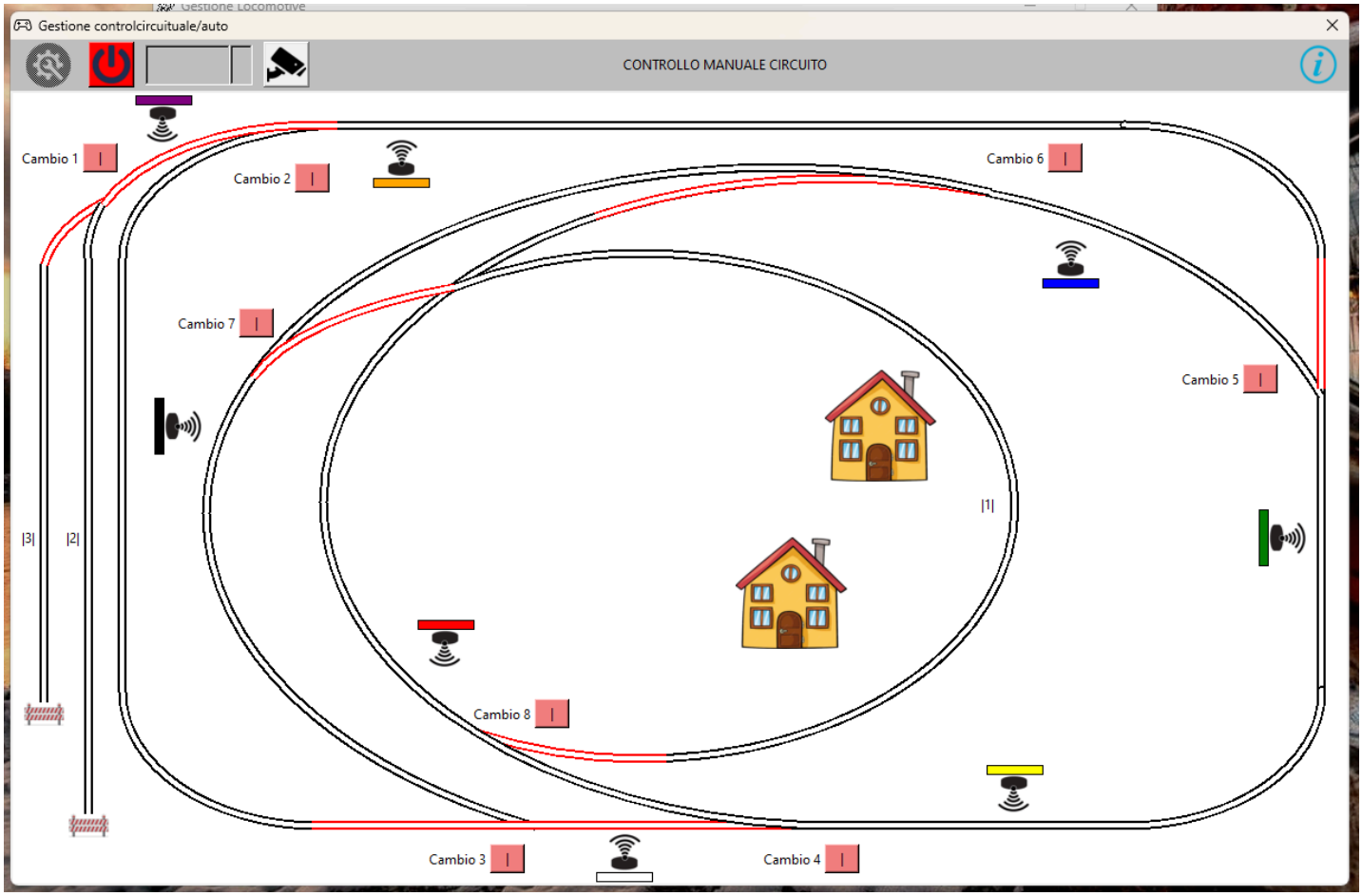


Figura 15: Pagina Monitoraggio Circuito

È inoltre possibile attivare la **modalità automatica** del sistema ferroviario. In questa modalità, il sistema controlla autonomamente le operazioni ferroviarie utilizzando algoritmi interni. Gli utenti possono monitorare l'andamento del sistema e intervenire manualmente se necessario.
L'unica **vincolo** per accedere ed attivarla è prima di tutto **avviare la modalità tramite il bottone di avvio o con il tasto 'o' da tastiera**.

Nel caso di dubbi è disponibile la **sezione informazioni tramite tasto 'i' da tastiera** o premendo il bottone blu.

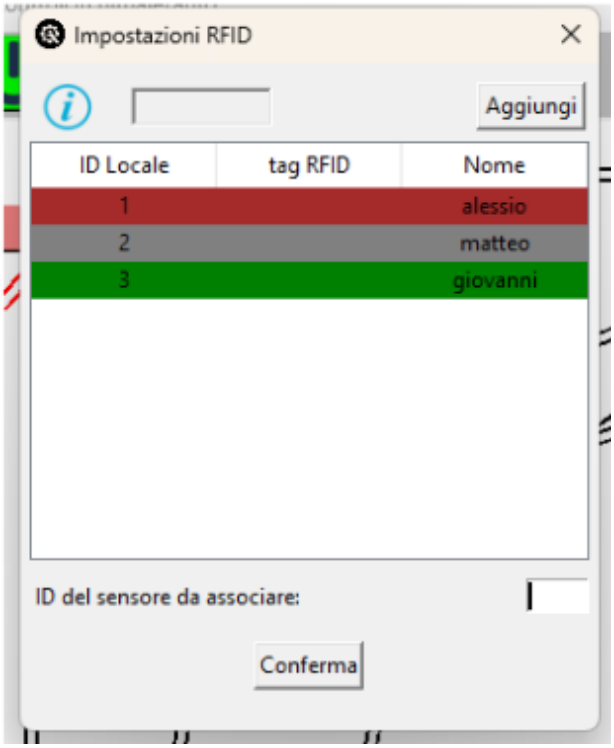


Figura 16: Pagina Impostazioni del monitoraggio circuito

Dopodiché è necessario **registrare le locomotive**, il procedimento è semplice, bisogna far **passare il tag RFID della locomotiva sotto qualsiasi sensore** del circuito, il codice apparirà nella casella posizionata sopra la tabella e sarà possibile **decidere a chi assegnarlo e poi confermare.**
Nel caso di dubbi anche in questa pagina è disponibile la **sezione informazioni tramite tasto 'i'** da tastiera o premendo il bottone blu.
Nel caso in cui ci si sia dimenticati di **aggiungere delle locomotive**, è possibile farlo anche da questa pagina, tramite il **bottone aggiungi o premendo '+' da tastiera**.

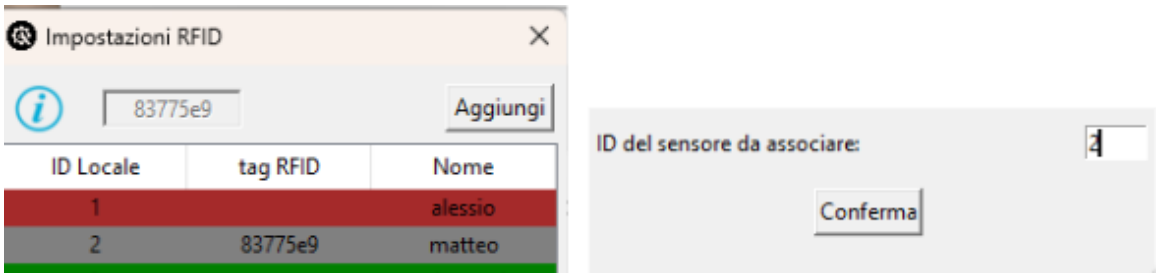


Figura 17: Inserimento di un tag nella pagina Impostazioni del monitoraggio circuito

Tornando alla pagina di monitoraggio del circuito, è possibile **avviare**, come detto in precedenza, **la visione della videocamera puntata sul circuito tramite il tasto 'v' da tastiera o premendo il bottone che ha come icona una telecamera.**
Il risultato è una finestra che può essere ingrandita a seconda dello schermo.

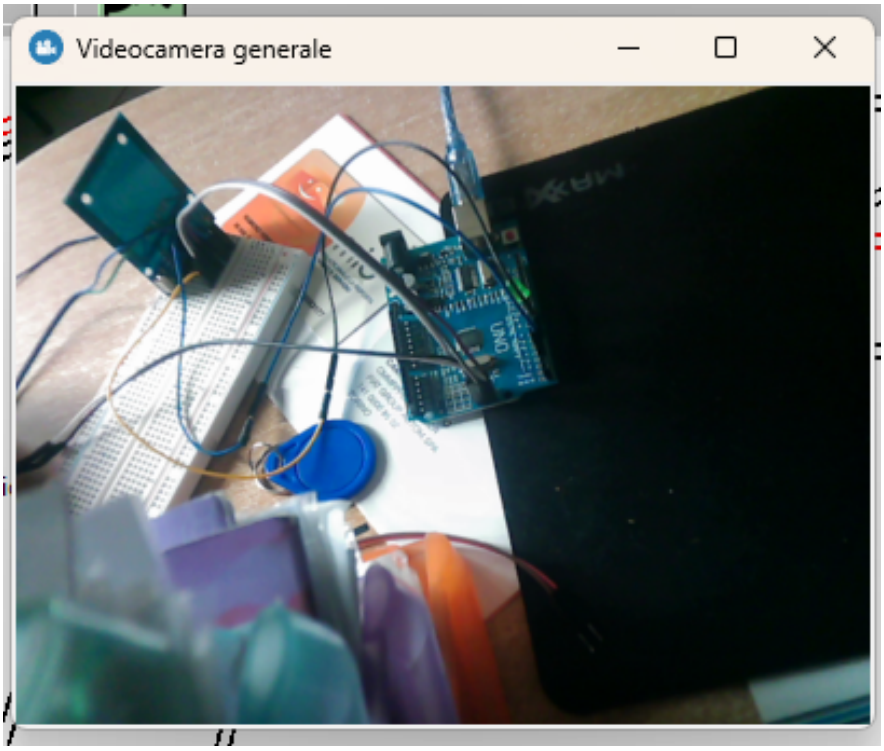


Figura 18: Pagina della videocamera

7.8 Documentazione e Contributi

È possibile visionare la **documentazione** completa in questo link:

- Documentazione tecnica GUI [5]

8 Sistema di Automazione

L'automazione del circuito ferroviario è stata realizzata utilizzando **relè e tecnologia RFID**. Questa sezione descrive come i componenti hardware e software lavorano insieme per **automatizzare il controllo delle locomotive e degli scambi e come è stato progettato l'algoritmo di controllo.**
Attualmente, questa parte del progetto è ancora **in fase di sviluppo** e può presentare alcune imperfezioni nel funzionamento.

8.1 Componenti hardware & software

Le **componenti hardware e software** sono le stesse implementate nel sistema manuale, la modifica principale è l'utilizzo appunto dell'**algoritmo di Controllo**.

8.2 Algoritmo di Controllo

L' algoritmo di controllo è il **cuore del sistema di automazione**, responsabile dell'**elaborazione delle informazioni** provenienti dai sensori e del **controllo degli attuatori.**
Attualmente, questo algoritmo è in **fase di ottimizzazione** e potrebbe non funzionare ancora perfettamente.

8.2.1 Acquisizione dei Dati dai Sensori RFID

L'acquisizione dei dati dai Sensori RFID funziona tramite:

- **Lettura dei Tag RFID:**
  Gli algoritmi **leggono continuamente i dati dei tag RFID rilevati** lungo il circuito. Ogni lettura contiene informazioni sulla locomotiva dalle quali si ricavano velocità, direzione e quindi posizione.
- **Gestione dei Dati:**
  I dati acquisiti vengono memorizzati in una struttura FIFO (First In, First Out) utilizzando la **libreria Queue**, assicurando che le **letture vengano processate nell'ordine corretto.**

8.2.2 Logica di Controllo degli Scambi

La logica di controllo degli scambi comprende:

- **Determinazione della Direzione:**
  Basandosi sulla posizione corrente delle locomotive e la **destinazione prevista**, gli algoritmi calcolano la **direzione corretta degli scambi.**
- **Esecuzione del Cambio di Scambio:**
  Utilizzando i relè, gli algoritmi inviano comandi per **cambiare la posizione degli scambi**, garantendo che le locomotive seguano il percorso corretto.

8.2.3 Prevenzione delle Collisioni

La Prevenzione delle collisioni comprende:

- **Ricerca dei punti critici:**
  L'algoritmo quando viene avviato **assegna in maniera casuale il percorso ad ogni locomotiva**, dopodiché **effettua il calcolo dei punti critici** che non è altro che l'**intersezione dei percorsi assegnati.**
  In questo modo si monitorano solo i punti critici mentre gli altri possono essere ignorati.
- **Azioni di Sicurezza:**
  Se una locomotiva prova ad andare in un punto in cui è presente un'altra locomotiva, verrà attivato il deviatoio e il **percorso iniziale sarà deviato.**

Codice del Calcolo dei punti Critici:

```
#Ritorna un vettore che contiene i percorsi liberi in quel momento
    def show_percorsi_liberi(self,id):
      return list(set(data.LRoutes.keys()) - set(data.percorsi_assegnati)) if
data.locomotives_data[id]['Direzione'] == 0 else list(set(data.RRoutes.keys()) -
set(data.percorsi_assegnati))

    #Ritorna l'intersezione tra i percorsi assegnati
    def trova_criticita(self):
        return set(data.LRoutes[data.percorsi_assegnati[0]]) &
set(data.RRoutes[data.percorsi_assegnati[-1]])

    def scegli_percorso(self, id):
        percorsi_liberi = self.show_percorsi_liberi(id)
        percorso_scelto = random.choice(percorsi_liberi)
        print("il percorso scelto è: " +str(percorso_scelto))
        data.percorsi_assegnati.append(percorso_scelto)

        data.locomotives_data[id]['Percorso'] = percorso_scelto
        if id ≠ 0:
            data.criticita = self.trova_criticita()
```

8.3 Modalità Automatica

La modalità automatica del sistema di controllo ferroviario permette una **gestione autonoma del circuito, migliorando l'efficienza e la sicurezza delle operazioni.**
Sebbene in fase di ottimizzazione, l'algoritmo potrebbe ancora presentare qualche imperfezione.

8.3.1 Attivazione della Modalità Automatica

L'Attività della modalità Automatica include:

- **Avvio dalla GUI:**
  Gli utenti possono **attivare la modalità automatica** tramite un semplice click nell'**interfaccia grafica dalla pagina circuit window.**
  Una volta attivata, il sistema prende il controllo delle locomotive e degli scambi.
  È comunque possibile intervenire, in caso, a scambiare i deviatoi.

- **Monitoraggio in Tempo Reale:**
  La GUI **visualizza in tempo reale la posizione delle locomotive e lo stato degli scambi**, permettendo agli utenti di monitorare l'andamento del circuito.

Codice dei dati relativi ai sensori e agli scambi:

```
#Creazione in memoria dei deviatoi
Turnouts = {
    "Cambio 1":   [False,"","",""],
    "Cambio 2":   [False,"","",""],
    "Cambio 3":   [False,"","",""],
    "Cambio 4":   [False,"","",""],
    "Cambio 5":   [False,"","",""],
    "Cambio 6":   [False,"","",""],
    "Cambio 7":   [False,"","",""],
    "Cambio 8":   [False,"","",""]
}

#creazione in memoria dei sensori
Sensors = {
    "Sensore 1":   ["","","","","purple"],
    "Sensore 2":   ["","","","","orange"],
    "Sensore 3":   ["","","","","white"],
    "Sensore 4":   ["","","","","yellow"],
    "Sensore 5":   ["","","","","green"],
    "Sensore 6":   ["","","","","blue"],
    "Sensore 7":   ["","","","","black"],
    "Sensore 8":   ["","","","","red"]
}
```

8.3.2 Operazioni Automate

Le operazioni Automate sono:

- **Controllo delle Locomotive:**
  La **velocità e la direzione delle locomotive vengono gestite automaticamente dagli algoritmi**, basandosi sui dati in ingresso dai sensori RFID.
- **Gestione degli Scambi:**
  Gli scambi vengono **aperti o chiusi**, in modo autonomo, assicurando che le **locomotive seguano il percorso previsto** senza bisogno di intervento manuale.
- **Prevenzione degli Incidenti:**
  Gli algoritmi di prevenzione delle collisioni operano **costantemente**, garantendo un **funzionamento sicuro e senza interruzioni del circuito.**

Codice del controllo del punto critico:

```
#   Questa funzione controlla che i percorsi adiacenti a quelli toccati dal treno,
nella direzione in cui sta andando siano liberi
    def control(self,Turnout,direzione,natural_link):

        for item in data.Adjacent_Turnouts[Turnout][direzione]:
#Se nel sensore c'è qualcosa, attiva lo scambio, almeno che non sia passivo,
#in quel caso deve fermare il treno e poi farlo ripartire quando passa
————————————————————————————————————————————————
            if data.Sensors["Sensore {}".format(item)][direzione+1] ≠ "" :
                #-1 vuoldire cambio passivo
                if natural_link == -1:
                    print("STOP")

                #Assegno il percorso che è occupato.
                data.root_occupied = item
                return natural_link==data.root_occupied

        return False
```

8.4 Test e Ottimizzazione

Dopo l'implementazione iniziale, il sistema di automazione è stato **sottoposto a rigorosi test** per verificarne il **funzionamento e ottimizzarne le prestazioni.**
Tuttavia, essendo in fase di sviluppo, potrebbe ancora presentare alcune **criticità**.

8.4.1 Procedure di Test

Le procedure di Test sono:

- **Test Funzionali:**
  Verifica del **corretto funzionamento** di tutte le funzionalità del sistema, inclusa la lettura dei sensori, il controllo degli scambi e la gestione delle locomotive.

8.4.2 Ottimizzazione

L'Ottimizzazione consiste nel:

- **Miglioramento delle Prestazioni:**
  Ottimizzazione dell'algoritmo di controllo e degli sketch di arduino UNO per garantire una **risposta rapida e senza ritardi**.
- **Risoluzione di Problemi:**
  Correzione di eventuali **bug** riscontrati durante i test e **miglioramento della stabilità complessiva** del sistema.

8.5 Considerazioni Finali

Nonostante le numerose sfide affrontate durante lo **sviluppo e la fase attuale di ottimizzazione**, il sistema è **progettato per essere robusto, efficiente e user-friendly**, con ampie possibilità di **estensione e miglioramento futuro.**

9 Sviluppi futuri

In questa sezione sono riportate le idee future che si vogliono implementare **dopo la conclusione del sistema di Automazione**.

9.1 Altri Miglioramenti

Gli altri miglioramenti considerati negli sviluppi futuri sono:

- **Installazione di videocamere per ogni locomotiva**
  Questo miglioramento è stato pensato e parzialmente integrato nella versione di test dell'applicazione.
  Installare delle videocamere su ogni locomotiva e gestire le immagini dall'applicazione è uno degli **obiettivi da raggiungere**.

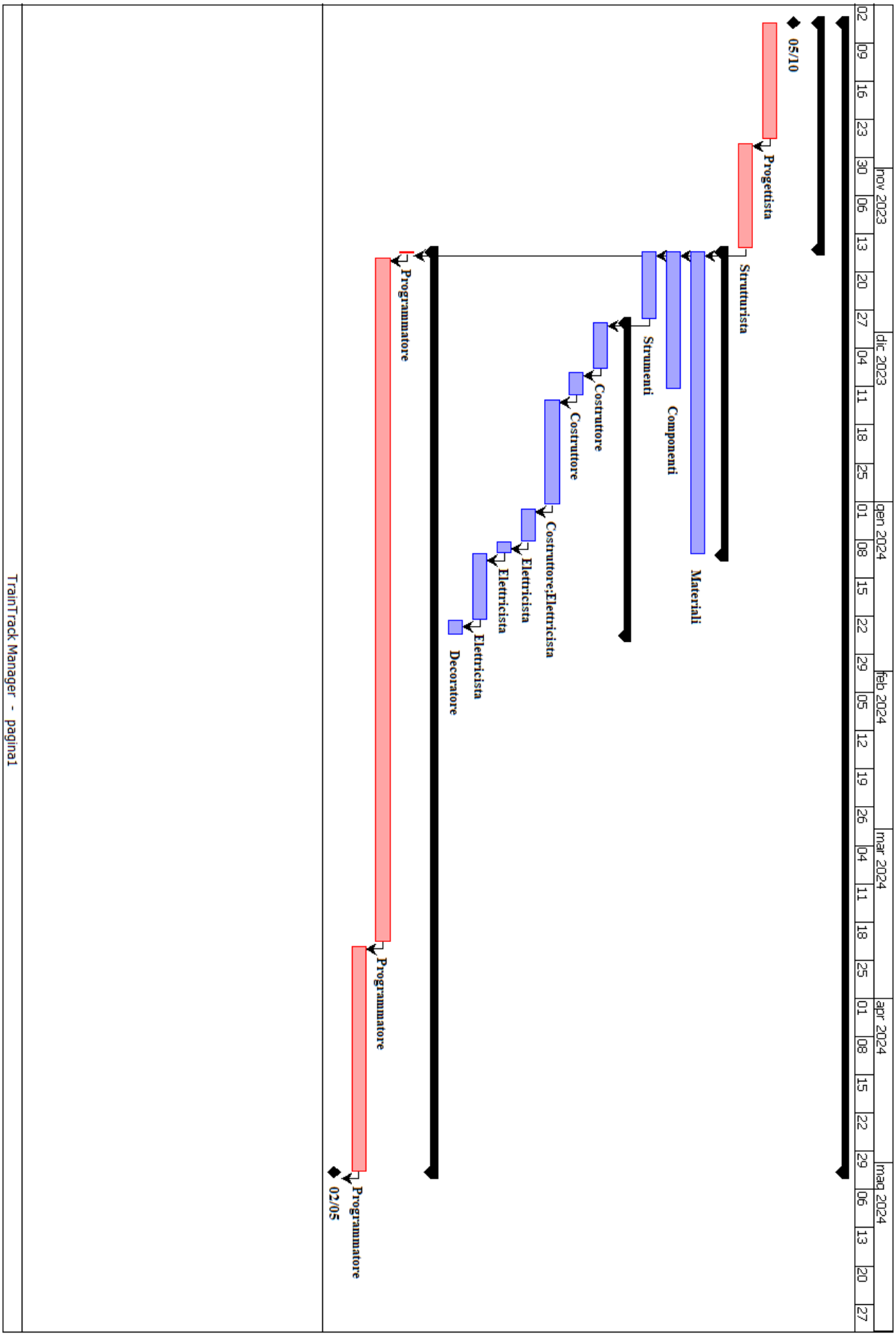
- **Controllo da remoto**
  Questa opzione intende sostituire gli arduino utilizzati con arduino **wifi**, come l'arduino r4 wifi, o in caso non midificasse le prestazioni utilizzare dei moduli **esp-32**, in modo da poter **avviare e controllare il sistema da remoto**, magari con un interfaccia creata appositamente.

- **Scalabilità dei circuiti costruibili sulla GUI**
  Un altro progetto è quello di ampliare la scelta per l'utente dandogli la possibilità di **creare e modificare il proprio disegno di circuito** e di **poterlo salvare per utilizzi futuri**. Questa modifica permetterebbe di **utilizzare l'applicazione per qualunque disegno di circuito.**

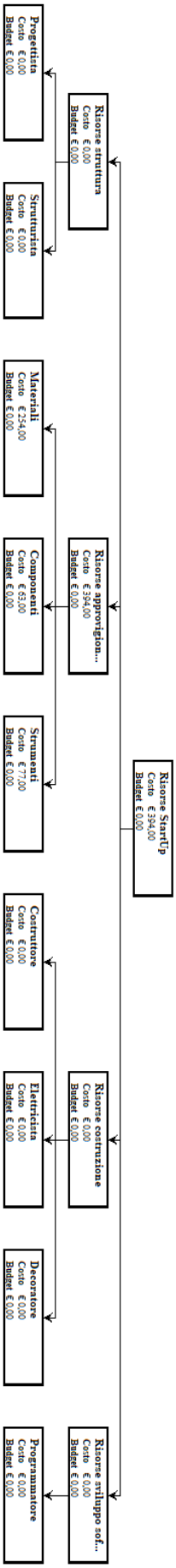
10 Diagramma di Gantt

		WBS	Nome	Durata	Data di Avvio	Data di chiusura	Predecessori	Nome risorsa
1		**1**	**StartUp Circuito Locomotive**	**151 giorni**	**05/10/23 8.00**	**02/05/24 17.00**		
2		**1.1**	**Struttura**	**30 giorni**	**05/10/23 8.00**	**15/11/23 17.00**		
3		1.1.1	Inizio	0 giorni	05/10/23 8.00	05/10/23 8.00		
4		1.1.2	Progettazione generale	16 giorni	05/10/23 8.00	26/10/23 17.00		Progettista
5		1.1.3	Ricerca informazioni sui materiali da utilizzare	14 giorni	27/10/23 8.00	15/11/23 17.00	4	Strutturista
6		**1.2**	**Approvigionamento**	**40 giorni**	**16/11/23 8.00**	**10/01/24 17.00**		
7		1.2.1	Materiali	40 giorni	16/11/23 8.00	10/01/24 17.00	5	Materiali
8		1.2.2	Componenti	17,5 giorni	16/11/23 8.00	11/12/23 13.00	5	Componenti
9		1.2.3	Strumenti	9 giorni	16/11/23 8.00	28/11/23 17.00	5	Strumenti
10		**1.3**	**Costruzione**	**41,2 giorni**	**29/11/23 8.00**	**25/01/24 9.36**		
11		1.3.1	Costruire il tavolo	7 giorni	29/11/23 8.00	07/12/23 17.00	9	Costruttore
12		1.3.2	Predisporre il tavolo	3 giorni	08/12/23 8.00	12/12/23 17.00	11	Costruttore
13		1.3.3	Creare il circuito	14 giorni	13/12/23 8.00	01/01/24 17.00	12	Costruttore;Elettricista
14		1.3.4	Installare componenti	4,5 giorni	02/01/24 8.00	08/01/24 13.00	13	Elettricista
15		1.3.5	Comporre la centralina	2 giorni	08/01/24 13.00	10/01/24 13.00	14	Elettricista
16		1.3.6	Test componenti elettrici	8,2 giorni	10/01/24 13.00	22/01/24 14.36	15	Elettricista
17		1.3.7	Decorare	2,5 giorni	22/01/24 14.36	25/01/24 9.36	16	Decoratore
18		**1.4**	**Sviluppo software**	**121 giorni**	**16/11/23 8.00**	**02/05/24 17.00**		
19		1.4.1	Creazione dell'ambiente di sviluppo	1 giorno	16/11/23 8.00	16/11/23 17.00	5	Programmatore
20		1.4.2	Sviluppo codice	90 giorni	17/11/23 8.00	21/03/24 17.00	19	Programmatore
21		1.4.3	Testing e debugging	30 giorni	22/03/24 8.00	02/05/24 17.00	20	Programmatore
22		1.4.4	Fine	0 giorni	02/05/24 17.00	02/05/24 17.00	21	

TrainTrack Manager


nov 2023
dic 2023
gen 2024
feb 2024
mar 2024
apr 2024
mag 2024
02 09 16 23 30 06 13 20 27 04 11 18 25 01 08 15 22 29 05 12 19 26 04 11 18 25 01 08 15 22 29 06 13 20 27
05/10
Progettista
Strutturista
Materiali
Componenti
Strumenti
Costruttore
Costruttore
Costruttore;Elettricista
Elettricista
Elettricista
Elettricista
Decoratore
Programmatore
Programmatore
Programmatore
02/05
TrainTrack Manager - pagina1

11 WBS


Risorse StartUp
Costo € 394,00
Budget € 0,00
Risorse struttura
Costo € 0,00
Budget € 0,00
Progettista
Costo € 0,00
Budget € 0,00
Strutturista
Costo € 0,00
Budget € 0,00
Risorse approvigion...
Costo € 394,00
Budget € 0,00
Materiali
Costo € 324,00
Budget € 0,00
Componenti
Costo € 65,00
Budget € 0,00
Strumenti
Costo € 7,00
Budget € 0,00
Risorse costruzione
Costo € 0,00
Budget € 0,00
Costruttore
Costo € 0,00
Budget € 0,00
Elettricista
Costo € 0,00
Budget € 0,00
Decoratore
Costo € 0,00
Budget € 0,00
Risorse sviluppo sof...
Costo € 0,00
Budget € 0,00
Programmatore
Costo € 0,00
Budget € 0,00

12 Tabelle dei Costi

In queste tabelle sono elencati i **costi relativi al progetto dal momento della partenza**, tutto cio che non è citato in queste tabelle era materiale di cui ero gia in possesso in precedenza.

Tabella 1: Tabella deti Costi derivanti da ordini su Amazon

Tipo	**Descrizione**	**Quantità**	**Data**	**Prezzo(€)**
Componente	decoder	3	11 ott 2023	**45**
Materiale	scambi	3	14 ott 2023	**57**
Componente	fotoaccoppiatori	50	18 nov 2023	**8**
Strumento	Lianshi 18V 5A 90W Alimentatore	1	19 dic 2023	**22**
Materiale	Cavo Elettrico, Cavo Awg 30 15m	1	1 gen 2024	**16**
Materiale	Single Row PCB Board Femmina	1	5 gen 2024	**12**
Materiale	Connettore Blocco Terminale	1	5 gen 2024	**13**
Totale Amazon	Somma del totale			**173**

Tabella 2: Tabella deti Costi derivanti da ordini su Ebay

Tipo	**Descrizione**	**Quantità**	**Data**	**Prezzo(€)**
Componente	presa dcc per locomotive	3	13 ott 2023	**10**
Materiale	scambi	7	15 ott 2023	**33**
Materiale	relè 12 volt	10	19 nov 2023	**45**
Totale Ebay	Somma del totale			**88**

Tabella 3: Tabella deti Costi derivanti da ordini su Arduino.cc

Tipo	**Descrizione**	**Quantità**	**Data**	**Prezzo(€)**
Strumento	motor shield rev 3	1	21 ott 2023	**30**
Strumento	Arduino Mega 2560	1	15 nov 2023	**25**
Totale Arduino.cc	Somma del totale			**55**

Tabella 4: Tabella deti Costi derivanti da ordini su TEMU

Tipo	**Descrizione**	**Quantità**	**Data**	**Prezzo(€)**
Materiale	ordine 24 dicembre	1	24 dic 2023	**23**
Materiale	ordine 28 dicembre	1	28 dic 2023	**8**
Materiale	ordine 2 gennaio	1	2 gen 2024	**8**
Materiale	ordine 21 gennaio	1	21 gen 2024	**17**
Materiale	ordine 22 gennaio	1	22 gen 2024	**22**
Totale TEMU	Somma deil totale			**78**

Tabella 5: Tabella deti Costi totali sommati

Totale	Somma del totale delle tabelle precedenti	**394 €**

Riferimenti

[1] «Documentazione tecnica dei circuiti elettrici». [Online]. Disponibile su: https://docs.google.com/document/d/1AeFnw-B1Prgavqx18H5ri2PBlTm2iWFwRN4Kjna7XKo/edit?usp=sharing

[2] «Arduino DCCpp Library». [Online]. Disponibile su: https://github.com/DccPlusPlus/BaseStation

[3] «Arduino RFID Library for MFRC522». [Online]. Disponibile su: https://github.com/miguelbalboa/rfid

[4] «TrainTrack Manager». [Online]. Disponibile su: https://github.com/alexein1001/translating

[5] «Documentazione tecnica GUI». [Online]. Disponibile su: https://www.traintrackdocs.altervista.org/